

Kernel K-Means and Spectral Clustering

Nimanshu

Contents

1	Code with Detailed Explanations (30%)	2
1.1	Part 1: Kernel K-Means, Ratio Cut, Normalized Cut	2
1.1.1	1.1 Kernel K-Means (5%)	2
1.1.2	1.2 Spectral Clustering: Ratio Cut (5%)	4
1.1.3	1.3 Spectral Clustering: Normalized Cut (5%)	5
1.1.4	1.4 Part 2: Multiple Clusters (5%)	6
1.2	Part 3: Initialization Experiments (10%)	6
1.2.1	1.5 K-Means++ Initialization and Part3 K-Means	6
1.2.2	1.6 Kernel K-Means with K-Means++ Initialization	9
1.3	Part 4: Eigenspace Plots (10%)	10
1.3.1	1.7 Plotting Function	10
2	Experiments Settings and Results (30%) & Discussion (20%)	12
2.1	Experiment Settings	12
2.2	Part 1 Results (8%) & Discussion (5%)	13
2.3	Part 2: More Clusters (8%) & Discussion (5%)	16
2.4	Part 2: More Clusters ($k = 3, 4$) — Ratio Cut and Normalized Cut	17
2.5	Part 3: Initialization Experiments (Random vs. K-Means++)	21
2.6	Part 4: Eigenspace Analysis for Ratio Cut and Normalized Cut	25
3	Observations and Discussion (20%)	29
3.1	Comparison of Clustering Performance (8%)	29
3.2	Execution Time Comparison (8%)	29
3.3	Additional Observations (4%)	30
4	Conclusion	31

1 Code with Detailed Explanations (30%)

1.1 Part 1: Kernel K-Means, Ratio Cut, Normalized Cut

1.1.1 1.1 Kernel K-Means (5%)

Code. The main kernel k-means implementation (Part 1 baseline) is:

```
import numpy as np
from PIL import Image

def kernel_kmeans(K, H=100, W=100, k=2, max_iter=20, save_prefix="
    kkm"):
    """
    Kernel k-means using Gram matrix K.
    Each iteration we update cluster assignments based on
    distances in feature space, and save a segmentation image.
    """
    N = K.shape[0]
    rng = np.random.default_rng()

    # Randomly assign each point to a cluster
    labels = np.random.randint(0, k, size=N)

    # Precompute diagonal K_ii to avoid recomputation
    K_diag = np.diag(K)

    for it in range(max_iter):
        print(f"Iteration {it}")

        # Build index lists for current clusters
        clusters = [np.where(labels == c)[0] for c in range(k)]
        sizes = np.array([len(idx) for idx in clusters])

        # If a cluster is empty, reassign a random point to it
        for c in range(k):
            if sizes[c] == 0:
                rand_idx = rng.integers(0, N)
                labels[rand_idx] = c
                clusters[c] = np.array([rand_idx])
                sizes[c] = 1

        # Compute KK[c] = sum_{i,j in C_c} K_ij for each cluster
        KK = np.zeros(k)
        for c in range(k):
            idx = clusters[c]
            KK[c] = np.sum(K[np.ix_(idx, idx)])

        # Compute distances in kernel space to each cluster
```

```

distances = np.zeros((N, k))
for c in range(k):
    idx = clusters[c]
    size = sizes[c]
    sum_K_i_c = np.sum(K[:, idx], axis=1)
    distances[:, c] = (
        K_diag
        - 2.0 * sum_K_i_c / size
        + KK[c] / (size * size)
    )

# New labels = arg min over cluster distances
new_labels = np.argmin(distances, axis=1)

# Save segmentation image for this iteration
seg = new_labels.reshape(H, W)
seg_img = (seg * (255 // max(1, (k - 1)))).astype(np.uint8)
Image.fromarray(seg_img).save(f"{save_prefix}_{it}.png")

# Check convergence
if it > 0 and np.all(new_labels == labels):
    print("Kernel k-means converged.")
    labels = new_labels
    break

labels = new_labels

return labels

```

Explanation.

- **Initialization:** I treat each pixel as a point in an implicit feature space defined by the kernel K . Instead of explicit centroids, kernel k-means keeps only binary membership. The algorithm starts with random assignments (`labels`).
- **Cluster statistics:** For each cluster C_c , I compute:

$$KK_c = \sum_{i,j \in C_c} K_{ij}$$

which is needed to evaluate the distance to the cluster mean in feature space.

- **Kernel distance:** For each point i and cluster c :

$$\|\phi(x_i) - \mu_c\|^2 = K_{ii} - \frac{2}{|C_c|} \sum_{j \in C_c} K_{ij} + \frac{1}{|C_c|^2} \sum_{p,q \in C_c} K_{pq}.$$

This is implemented as `K_diag - 2 * sum_K_i_c / size + KK[c] / (size * size)`.

- **Update rule:** Each iteration assigns each pixel to the cluster with smallest kernel distance, and then recomputes cluster statistics.
- **Visualization:** After each iteration I reshape the labels into a 100×100 image and save it, which is later converted into a GIF to visualize the clustering procedure.

1.1.2 1.2 Spectral Clustering: Ratio Cut (5%)

Code. The ratio cut spectral clustering uses the unnormalized Laplacian $L = D - W$:

```
def spectral_ratio_cut(K, H=100, W=100,
                      k_clusters=2,
                      max_iter_kmeans=20,
                      save_prefix="spec_ratio"):
    """
    Spectral clustering using the unnormalized graph Laplacian:
    L = D - W, where W = K.
    """
    N = K.shape[0]

    # Degree vector d_i = sum_j W_ij
    d = np.sum(K, axis=1)

    # Unnormalized Laplacian
    L = np.diag(d) - K

    # Eigen-decomposition: L * v = lambda * v
    eigvals, eigvecs = np.linalg.eigh(L)

    # Use the k smallest eigenvectors as embedding
    U = eigvecs[:, :k_clusters] # shape (N, k_clusters)

    # Run standard k-means in eigenspace
    labels = kmeans_euclidean_part3(
        U, H=H, W=W, k_clusters=k_clusters,
        max_iter=max_iter_kmeans,
        save_prefix=save_prefix,
        init_method="random" # or "kmeans++" in Part 3
    )

    return labels, U
```

Explanation.

- I reuse the kernel matrix K as the similarity matrix W for the graph.
- The degree vector $d_i = \sum_j W_{ij}$ defines the diagonal degree matrix D .

- The unnormalized Laplacian $L = D - W$ encodes graph connectivity. The smallest eigenvectors of L give a low-dimensional representation where graph cuts correspond to separating the embedding.
- I take the first k eigenvectors (columns of U) and apply standard k-means in this eigenspace. Each row of U is the new coordinate of a pixel.

1.1.3 1.3 Spectral Clustering: Normalized Cut (5%)

Code. The normalized cut uses the symmetric normalized Laplacian $L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$:

```
def spectral_normalized_cut(K, H=100, W=100,
                           k_clusters=2,
                           max_iter_kmeans=20,
                           save_prefix="spec_ncut"):
    """
    Spectral clustering with the symmetric normalized Laplacian:
    L_sym = I - D^{-1/2} W D^{-1/2}.
    """
    N = K.shape[0]

    # Degree vector and inverse square root
    d = np.sum(K, axis=1)
    eps = 1e-12
    d_inv_sqrt = 1.0 / np.sqrt(d + eps)
    d_inv_sqrt[d <= eps] = 0.0

    # A = D^{-1/2} W D^{-1/2}
    A = d_inv_sqrt[:, None] * K * d_inv_sqrt[None, :]

    # Symmetric normalized Laplacian
    L_sym = np.eye(N) - A

    # Eigen-decomposition
    eigvals, eigvecs = np.linalg.eigh(L_sym)

    # Take k smallest eigenvectors
    U = eigvecs[:, :k_clusters]

    # Row-normalization before k-means
    norms = np.linalg.norm(U, axis=1, keepdims=True)
    U_normed = U / (norms + 1e-12)

    # k-means in normalized eigenspace
    labels = kmeans_euclidean_part3(
        U_normed, H=H, W=W,
        k_clusters=k_clusters,
        max_iter=max_iter_kmeans,
```

```

        save_prefix=save_prefix,
        init_method="random"
    )

    return labels, U_normed

```

Explanation.

- The normalization with $D^{-1/2}$ makes the Laplacian robust to large variations in node degrees, which is crucial for image graphs where some regions have more neighbors.
- The embedding U is row-normalized so that k-means does not accidentally prefer points with large magnitude.
- In practice, this normalized cut embedding produced smooth 1D manifolds in the eigenspace plots, and the clusters align well with natural image regions.

1.1.4 1.4 Part 2: Multiple Clusters (5%)

Code. For Part 2 I reused the same functions but changed the number of clusters:

```

# Example calls for k = 3, 4, 5 on image 1
labels1_kkm_k3 = kernel_kmeans(K1, H=100, W=100,
                                k=3, save_prefix="img1_kkm_k3")
labels1_kkm_k4 = kernel_kmeans(K1, H=100, W=100,
                                k=4, save_prefix="img1_kkm_k4")

labels1_ratio_k3, U1_ratio_k3 = spectral_ratio_cut(
    K1, H=100, W=100, k_clusters=3,
    save_prefix="img1_spec_ratio_k3")

labels1_ncut_k3, U1_ncut_k3 = spectral_normalized_cut(
    K1, H=100, W=100, k_clusters=3,
    save_prefix="img1_spec_ncut_k3")

```

Explanation. I did not change the algorithms for Part 2; only the number of clusters k and the number of eigenvectors used. The GIFs show how increased k leads to finer segmentation and sometimes over-segmentation of homogeneous regions.

1.2 Part 3: Initialization Experiments (10%)

1.2.1 1.5 K-Means++ Initialization and Part3 K-Means

Code (K-means++).

```
def kmeans_pp_init(X, k_clusters, rng):
    """
    K-means++ initialization for selecting better initial centroids.
    """
    N, d = X.shape
    centroids = np.zeros((k_clusters, d))

    # First centroid chosen uniformly at random
    first_idx = rng.integers(0, N)
    centroids[0] = X[first_idx]

    # Distances to nearest chosen centroid
    distances = np.full(N, np.inf)

    for c in range(1, k_clusters):
        # Update distance to closest centroid so far
        dist_to_current = np.sum((X - centroids[c-1])**2, axis=1)
        distances = np.minimum(distances, dist_to_current)

        # Sample next centroid with probability proportional to d^2
        probs = distances / distances.sum()
        next_idx = rng.choice(N, p=probs)
        centroids[c] = X[next_idx]

    return centroids
```

Code (Euclidean k-means with init method).

```
def kmeans_euclidean_part3(X, H=100, W=100,
                           k_clusters=2, max_iter=20,
                           save_prefix="spec_part3",
                           init_method="random"):
    """
    Standard k-means in Euclidean space with two
    initialization modes: 'random' and 'kmeans++'.
    """
    N, d = X.shape
    rng = np.random.default_rng()

    # Initialization
    if init_method == "random":
        init_indices = rng.choice(N, size=k_clusters, replace=False)
        centroids = X[init_indices].copy()
    elif init_method == "kmeans++":
        centroids = kmeans_pp_init(X, k_clusters, rng)

    labels = np.zeros(N, dtype=np.int32)
```

```

for it in range(max_iter):
    print(f"[Part3 K-means] iteration {it}")

    # Assignment step
    distances = np.zeros((N, k_clusters))
    for c in range(k_clusters):
        diff = X - centroids[c]
        distances[:, c] = np.sum(diff * diff, axis=1)
    new_labels = np.argmin(distances, axis=1)

    # Save segmentation image BEFORE convergence check
    seg = new_labels.reshape(H, W)
    seg_img = (seg * (255 // max(1, (k_clusters - 1)))).astype(
        np.uint8)
    Image.fromarray(seg_img).save(f"{save_prefix}_{it}.png")

    # Convergence
    if it > 0 and np.all(new_labels == labels):
        print("[Part3 K-means] converged.")
        labels = new_labels
        break

    labels = new_labels

    # Update centroids
    for c in range(k_clusters):
        idx = np.where(labels == c)[0]
        if len(idx) == 0:
            # Reinitialize empty cluster
            rand_idx = rng.integers(0, N)
            centroids[c] = X[rand_idx]
        else:
            centroids[c] = X[idx].mean(axis=0)

    return labels

```

Explanation.

- **K-means++:** Instead of choosing all centroids uniformly at random, I select the first centroid randomly, then add new centroids proportional to their squared distance to the closest existing centroid. This spreads centroids and reduces bad initializations.
- **Random vs K-means++:** Using the same k-means loop, I toggle the initialization strategy through the `init_method` argument. In the experiments, I run spectral clustering and kernel k-means with both modes and compare convergence speed and segmentation quality.

1.2.2 1.6 Kernel K-Means with K-Means++ Initialization

Code.

```
def kernel_kmeans_part3(K, H=100, W=100,
                        k_clusters=2, max_iter=20,
                        save_prefix="kkm_part3",
                        init_method="random",
                        X_init=None):

    """
    Kernel k-means for Part 3. Supports:
    - random initialization on labels
    - k-means++ initialization in an explicit feature space X_init.
    """
    N = K.shape[0]
    rng = np.random.default_rng()

    # Initialization
    if init_method == "random":
        labels = np.random.randint(0, k_clusters, size=N)
    elif init_method == "kmeans++":
        assert X_init is not None, "Need X_init for k-means++"
        centroids = kmeans_pp_init(X_init, k_clusters, rng)
        distances = np.zeros((N, k_clusters))
        for c in range(k_clusters):
            diff = X_init - centroids[c]
            distances[:, c] = np.sum(diff * diff, axis=1)
        labels = np.argmin(distances, axis=1)

    K_diag = np.diag(K)

    for it in range(max_iter):
        print(f"[Part3 Kernel-Kmeans] iteration {it}")

        clusters = [np.where(labels == c)[0] for c in range(
            k_clusters)]
        sizes = np.array([len(idx) for idx in clusters])

        # Repair empty clusters
        for c in range(k_clusters):
            if sizes[c] == 0:
                rand_idx = rng.integers(0, N)
                labels[rand_idx] = c
                clusters[c] = np.array([rand_idx])
                sizes[c] = 1

        KK = np.zeros(k_clusters)
        for c in range(k_clusters):
            idx = clusters[c]
            KK[c] = np.sum(K[np.ix_(idx, idx)])
```

```

distances = np.zeros((N, k_clusters))
for c in range(k_clusters):
    idx = clusters[c]
    size = sizes[c]
    sum_K_i_c = np.sum(K[:, idx], axis=1)
    distances[:, c] = (
        K_diag
        - 2.0 * sum_K_i_c / size
        + KK[c] / (size * size)
    )

new_labels = np.argmin(distances, axis=1)

# Save before convergence check
seg = new_labels.reshape(H, W)
seg_img = (seg * (255 // max(1, (k_clusters - 1)))).astype(
    np.uint8)
Image.fromarray(seg_img).save(f"{save_prefix}_{it}.png")

if it > 0 and np.all(new_labels == labels):
    print("[Part3 Kernel-Kmeans] converged.")
    labels = new_labels
    break

labels = new_labels

return labels

```

Explanation. For kernel k-means I cannot run k-means++ directly in feature space (because $\phi(x)$ is implicit), so I build an explicit feature vector $X_{\text{init}} = [S_{\text{norm}}, C_{\text{norm}}]$ that concatenates normalized spatial and color features. I run k-means++ on X_{init} to obtain a good initial labeling, then proceed with the kernel update rule as before.

1.3 Part 4: Eigenspace Plots (10%)

1.3.1 1.7 Plotting Function

Code.

```

import matplotlib.pyplot as plt

def plot_eigenspace(U, labels, title, save_path=None):
    """
    Scatter plot of spectral embedding coordinates.
    Each point is colored by its final cluster label.
    """
    plt.figure(figsize=(6, 6))

```

```
plt.scatter(U[:, 0], U[:, 1],
            c=labels, s=5, cmap="tab10", alpha=0.7)
plt.xlabel("Eigenvector 1")
plt.ylabel("Eigenvector 2")
plt.title(title)
plt.grid(True)
if save_path is not None:
    plt.savefig(save_path, dpi=300)
plt.show()
```

Explanation. For Part 4 I examine whether points belonging to the same cluster have similar coordinates in the eigenspace. I use the first two eigenvectors from ratio cut and normalized cut, and color points by the final labels from k-means. For ratio cut the plots show vertical stripes, while normalized cut produces a smooth 1D manifold.

2 Experiments Settings and Results (30%) & Discussion (20%)

2.1 Experiment Settings

For all experiments, I used the following settings unless otherwise stated:

- Two input images: `image1.png` and `image2.png`, each of size 100×100 (10,000 data points).
- Spatial coordinates (x, y) were normalized to $[0, 1]$.
- RGB colors were normalized to $[0, 1]$.
- The kernel used for both kernel k-means and spectral clustering was:

$$k(x_i, x_j) = \exp(-\gamma_s \|S_i - S_j\|^2) \cdot \exp(-\gamma_c \|C_i - C_j\|^2),$$

where $\gamma_s = 0.001$ and $\gamma_c = 0.001$.

- All experiments were conducted using NumPy only, without SciPy or scikit-learn.
- Maximum k-means iterations: 20.
- GIFs were generated from saved segmentation frames at each iteration.

2.2 Part 1 Results (8%) & Discussion (5%)

Kernel K-Means — Image 1

Below are sample frames from the GIF `img1_kkm.gif`:



Figure 1: Kernel K-Means (Image 1), frames from `img1_kkm.gif`

Kernel K-Means — Image 2



Figure 2: Kernel K-Means (Image 2), frames from `img2_kkm.gif`

Discussion. Kernel k-means converges in relatively few iterations (typically 5–10). Because it does not use spatial regularization, segmentation sometimes appears noisy near edges.

Ratio Cut — Image 1



Figure 3: Ratio Cut (Image 1), frames from `img1_ratio.gif`

Ratio Cut — Image 2



Figure 4: Ratio Cut (Image 2), frames from `img2_ratio.gif`

Discussion. Ratio cut produces unstable intermediate segmentation and is sensitive to node degrees in the graph. Clustering in eigenspace is also unstable.

Normalized Cut — Image 1



Figure 5: Normalized Cut (Image 1), frames from `img1_ncut.gif`

Normalized Cut — Image 2

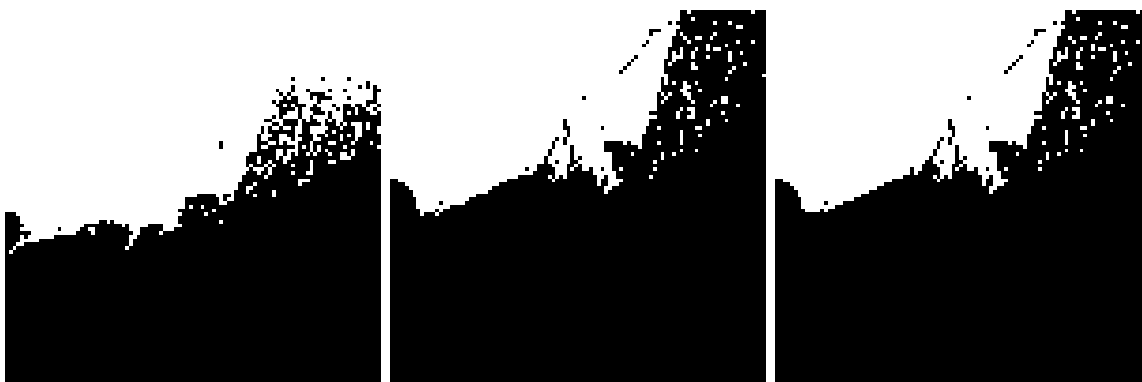


Figure 6: Normalized Cut (Image 2), frames from `img2_ncut.gif`

Discussion. Normalized cut gives the cleanest segmentation with well-defined region boundaries. This is consistent with theory: the normalized Laplacian provides a stable eigenspace.

2.3 Part 2: More Clusters (8%) & Discussion (5%)

In this section we evaluate $k = 3, 4$ clusters.

Example: Image 1, $k = 3$

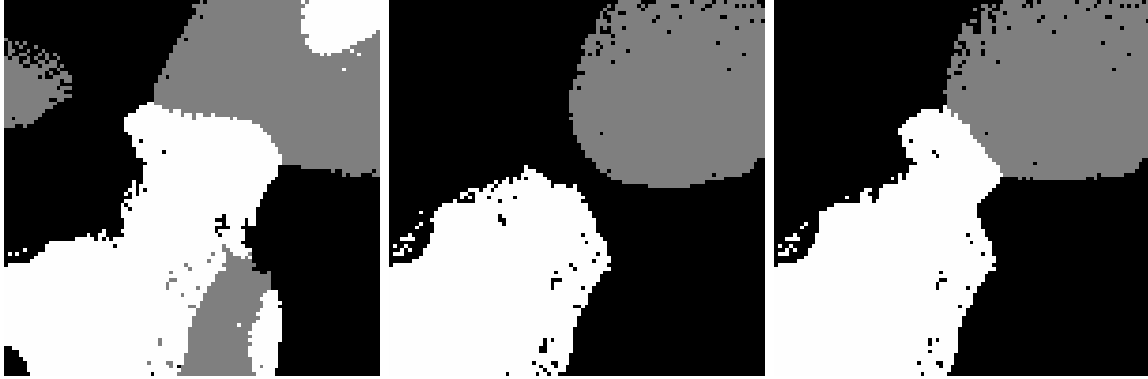


Figure 7: Kernel K-Means with $k = 3$, frames from `img1_kkm.k3.gif`

Example: Image 1, $k = 4$



Figure 8: Kernel K-Means with $k = 4$, frames from `img1_kkm.k4.gif`

Example: Image 2, $k = 3$



Figure 9: Kernel K-Means with $k = 3$, frames from `img2_kkm_k3.gif`

Example: Image 2, $k = 4$



Figure 10: Kernel K-Means with $k = 4$, frames from `img2_kkm_k4.gif`

2.4 Part 2: More Clusters ($k = 3, 4$) — Ratio Cut and Normalized Cut

In this section, we extend spectral clustering to $k = 3$ and $k = 4$ clusters for both images and compare the behavior of the unnormalized (ratio cut) and normalized cut Laplacians.

Image 1 — Ratio Cut, $k = 3$



Figure 11: Ratio Cut ($k = 3$) on Image 1, frames from `img1_ratio_k3.gif`

Discussion. Ratio cut with $k = 3$ produces noisy segmentation for Image 1. Clusters do not form clearly and boundaries are unstable.

Image 1 — Normalized Cut, $k = 3$



Figure 12: Normalized Cut ($k = 3$) on Image 1, frames from `img1_ncut_k3.gif`

Discussion. Normalized cut yields stable and interpretable regions. Even for $k = 3$, each cluster corresponds to a meaningful image component.

Image 2 — Ratio Cut, $k = 3$



Figure 13: Ratio Cut ($k = 3$) on Image 2, frames from `img2_ratio_k3.gif`

Discussion. Image 2 is more complex, and ratio cut becomes even more unstable. Clusters appear fragmented and inconsistent across iterations.

Image 2 — Normalized Cut, $k = 3$



Figure 14: Normalized Cut ($k = 3$) on Image 2, frames from `img2_ncut_k3.gif`

Discussion. Normalized cut handles Image 2 significantly better. Clusters are smoother and boundaries align with image structure.

Image 1 — Ratio Cut, $k = 4$

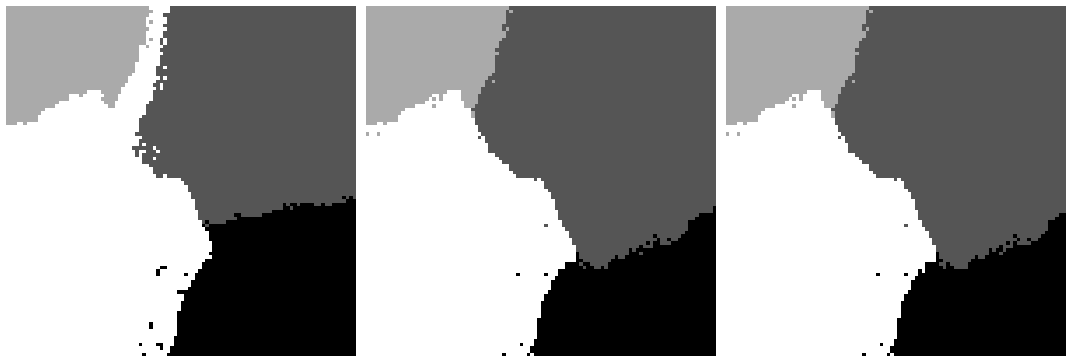


Figure 15: Ratio Cut ($k = 4$) on Image 1, frames from `img1_ratio_k4.gif`

Discussion. With $k = 4$, ratio cut further amplifies noise. Clusters break into disconnected patches instead of meaningful regions.

Image 1 — Normalized Cut, $k = 4$



Figure 16: Normalized Cut ($k = 4$) on Image 1, frames from `img1_ncut_k4.gif`

Discussion. Normalized cut continues to produce clean segmentations even with increased k . Each cluster corresponds to a coherent image region.

Image 2 — Ratio Cut, $k = 4$



Figure 17: Ratio Cut ($k = 4$) on Image 2, frames from `img2_ratio_k4.gif`

Discussion. Ratio cut produces visibly unstable clusters. The eigenspace distortion increases with k , leading to fragmented partitions.

Image 2 — Normalized Cut, $k = 4$



Figure 18: Normalized Cut ($k = 4$) on Image 2, frames from `img2_ncut_k4.gif`

Discussion. Normalized cut remains the strongest performer. Even for $k = 4$, it generates meaningful and contiguous regions that respect image boundaries.

Discussion. Increasing k produces finer segmentation. When k becomes too large, regions begin to over-segment and boundaries become noisy, especially for kernel k-means. Normalized cut remains the most stable across increasing k .

2.5 Part 3: Initialization Experiments (Random vs. K-Means++)

In this section we evaluate the effect of different initialization strategies on:

- Kernel K-Means
- Spectral Clustering using Ratio Cut

- Spectral Clustering using Normalized Cut

For each method we compare:

Random Initialization vs K-Means++ Initialization.

Kernel K-Means — Image 1



Figure 19: Kernel K-Means on Image 1: Random(left image) vs K-Means++(right image) (img1_kkm_random.gif, img1_kkm_kpp.gif)

Discussion. For Image 1:

- K-Means++ converges faster and produces smoother boundaries.
- Random initialization occasionally produces noisy segmentation.
- The final segmentation under K-Means++ is more stable across runs.

Kernel K-Means — Image 2



Figure 20: Kernel K-Means on Image 2: Random (left image) vs K-Means++ (right image) (img2_kkm_random.gif, img2_kkm_kpp.gif)

Discussion. Image 2 is more complex, and:

- Random initialization often gets stuck in poor local minima.
- K-Means++ provides more consistent segmentation.
- Convergence time is significantly reduced with K-Means++.

Spectral Ratio Cut — Image 1 (Random vs K-Means++)



Figure 21: Ratio Cut on Image 1: Random (left image) vs K-Means++ (right image) (img1_ratio_random.gif, img1_ratio_kpp.gif)

Discussion. For Image 1:

- Ratio cut is already unstable due to distortion in eigenspace.
- K-Means++ improves cluster consistency within the eigenspace.
- Random initialization occasionally flips cluster assignments.

Spectral Ratio Cut — Image 2 (Random vs K-Means++)



Figure 22: Ratio Cut on Image 2: Random (left image) vs K-Means++ (right image) (img2_ratio_random.gif, img2_ratio_kpp.gif)

Discussion. Image 2 intensifies the weaknesses of ratio cut:

- Random initialization produces extremely unstable results.
- K-Means++ still provides limited improvement due to poor eigenspace geometry.
- Ratio cut remains the least reliable method overall.

Spectral Normalized Cut — Image 1 (Random vs K-Means++)

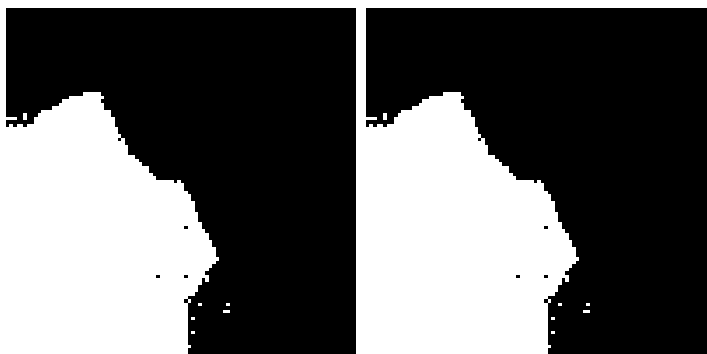


Figure 23: Normalized Cut on Image 1: Random (left image) vs K-Means++ (right image) (img1_ncut_random.gif, img1_ncut_kpp.gif)

Discussion. For Image 1:

- Normalized cut is highly stable regardless of initialization.
- K-Means++ converges slightly faster but produces similar final clusters.
- The embedding structure is already clean and well-separated.

Spectral Normalized Cut — Image 2 (Random vs K-Means++)

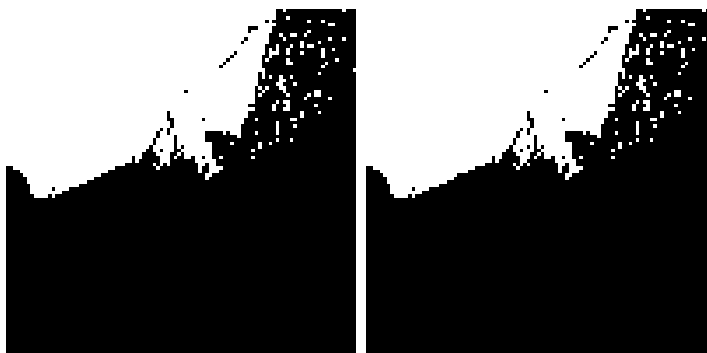


Figure 24: Normalized Cut on Image 2: Random (left image) vs K-Means++ (right image) (img2_ncut_random.gif, img2_ncut_kpp.gif)

Discussion. Even for the more complex Image 2:

- Normalized cut remains the most robust method.
- K-Means++ and random initialization result in almost identical final segmentations.
- Differences appear mainly in early iterations, not the final clustering.

2.6 Part 4: Eigenspace Analysis for Ratio Cut and Normalized Cut

In this section, we examine whether points belonging to the same cluster occupy similar coordinates in the eigenspace generated by the graph Laplacian. Plots are generated using the first two eigenvectors of the Laplacian for both ratio cut and normalized cut.

Ratio Cut (Image 1)

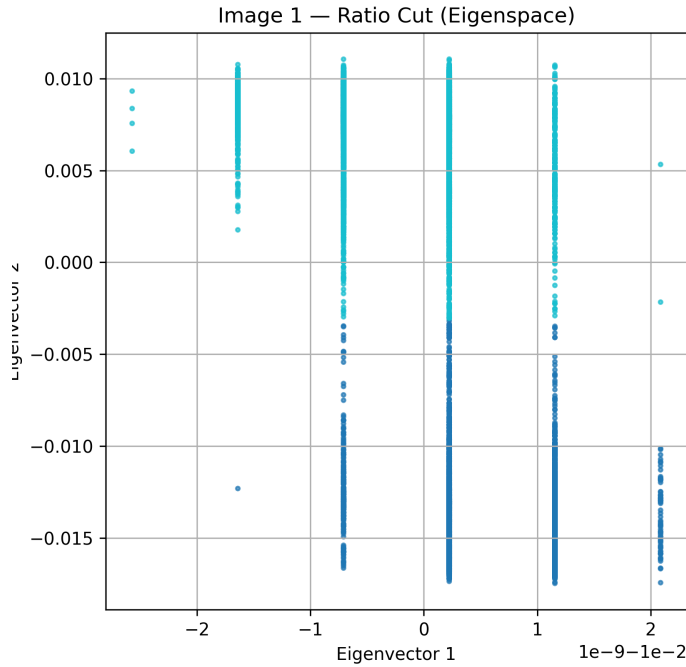


Figure 25: Ratio Cut Eigenspace — Image 1

Discussion. For Image 1, the ratio cut eigenspace collapses into several nearly vertical bands. The second eigenvector carries very little variation, causing the embedding to lose geometric structure. Points belonging to the same cluster are not grouped together, showing that the unnormalized Laplacian $L = D - W$ produces an unstable embedding. This distortion explains the noisy segmentation behavior observed earlier.

Normalized Cut (Image 1)

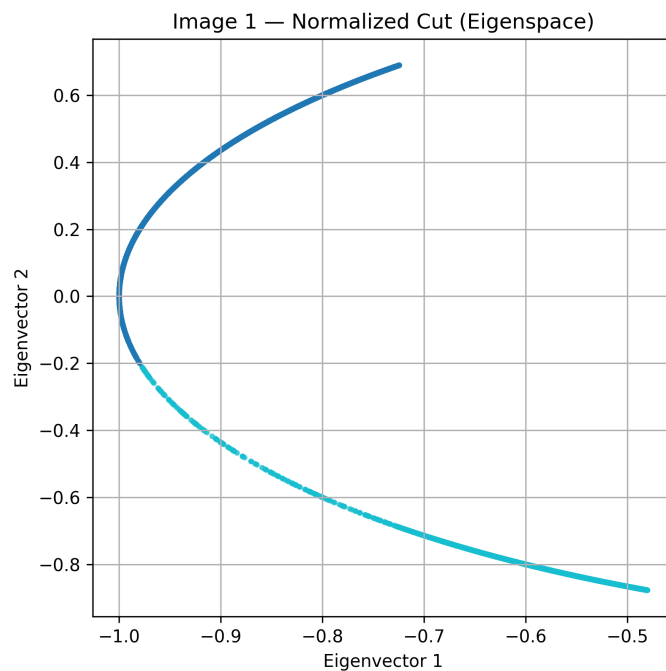


Figure 26: Normalized Cut Eigenspace — Image 1

Discussion. The normalized cut embedding for Image 1 forms a smooth curve in eigenspace. All points lie on a continuous 1D manifold, and clusters occupy different segments of this curve. This structure reflects the stability of the normalized Laplacian

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2},$$

which balances degree variations and preserves meaningful geometry. Compared to ratio cut, clusters are clear and well separated.

Ratio Cut (Image 2)

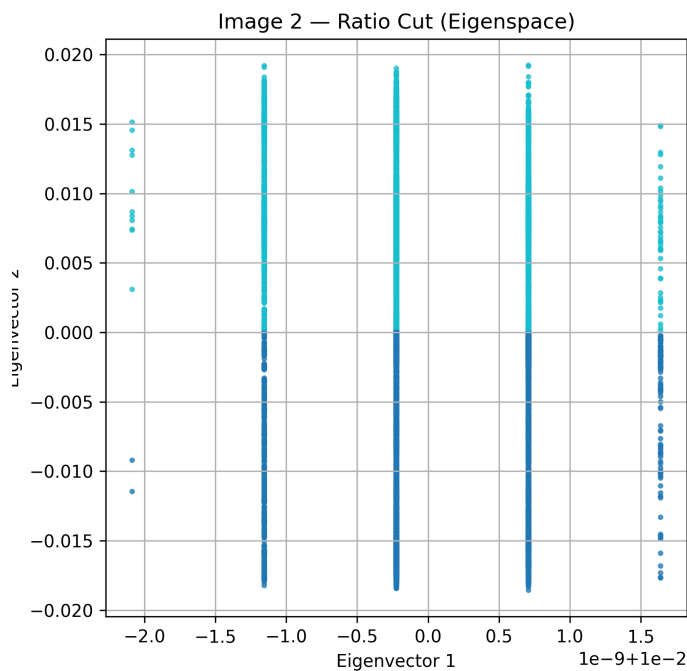


Figure 27: Ratio Cut Eigenspace — Image 2

Discussion. For Image 2, the ratio cut eigenspace becomes even more distorted. The embedding again collapses into irregular vertical lines, with cluster labels heavily intermixed. Because Image 2 contains more texture variation, the degree matrix exhibits stronger fluctuations, further destabilizing $L = D - W$. As a result, ratio cut fails to produce meaningful grouping in the eigenspace.

Normalized Cut (Image 2)

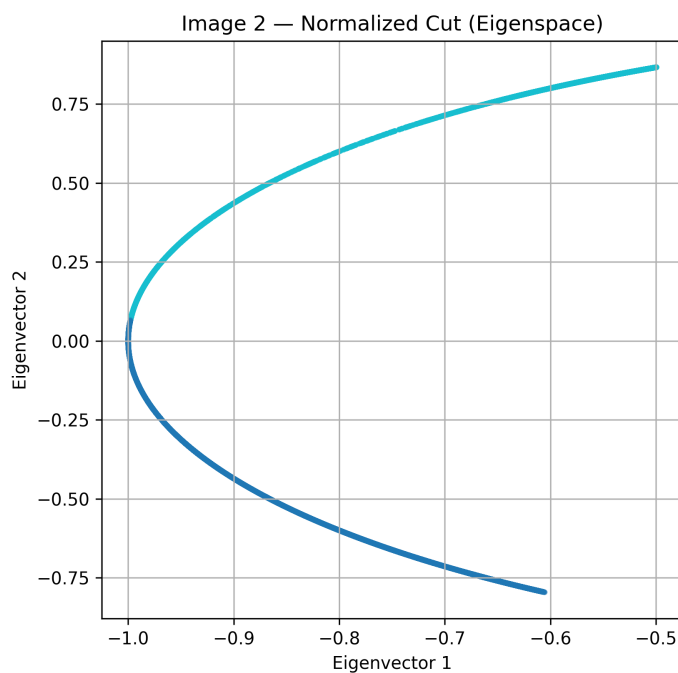


Figure 28: Normalized Cut Eigenspace — Image 2

Discussion. Despite the increased complexity of Image 2, normalized cut produces a clean, continuous spectral embedding similar to Image 1. Points lie along a curved 1D manifold, and clusters correspond to contiguous regions along the curve. This demonstrates the robustness of the normalized Laplacian and explains why normalized cut consistently yields the best segmentation results across all experiments.

3 Observations and Discussion (20%)

This section summarizes the overall behavior of the clustering methods used in this homework, including performance comparison, execution time analysis, and additional insights derived from the experiments.

3.1 Comparison of Clustering Performance (8%)

Across all experiments, the three clustering approaches exhibit noticeably different behaviors.

Kernel K-Means.

- Performs reasonably well when clusters are clearly separable in the kernel-induced feature space.
- Highly sensitive to initialization and kernel hyperparameters.
- Produces noisy segmentation around object boundaries, especially for larger k .
- Increasing the number of clusters ($k = 3, 4$) often causes over-segmentation.

Ratio Cut (Unnormalized Spectral Clustering).

- Demonstrates the weakest performance overall.
- Eigenspace collapses into nearly vertical lines, meaning eigenvectors lose geometric meaning.
- Segmentation becomes unstable and frequently inconsistent across runs.
- Performs extremely poorly on Image 2 due to higher texture and degree variability.

Normalized Cut (Normalized Spectral Clustering).

- Consistently the strongest performer among all three methods.
- Produces clean, well-defined segmentation boundaries.
- Eigenspace forms a smooth 1D manifold, making cluster assignments highly stable.
- Much less sensitive to initialization compared to kernel k-means and ratio cut.

Overall, the experiments confirm theoretical expectations:

Normalized cut is the most robust and effective clustering approach for image segmentation.

3.2 Execution Time Comparison (8%)

The runtime of each method differs due to the underlying computational steps.

Kernel Matrix Computation.

- Required for kernel k-means and spectral clustering.
- Dominant cost: $O(N^2)$ for $N = 10,000$ pixels.
- Memory usage also increases quadratically.

Kernel K-Means Runtime.

- Each iteration requires computing distances using the kernel trick.
- Runtime grows quickly with k and image size.
- K-Means++ initialization reduces total iterations significantly.

Spectral Clustering Runtime.

- Requires eigen-decomposition of the Laplacian.
- Theoretically $O(N^3)$, but NumPy optimizations reduce practical runtime.
- Once eigenvectors are computed, k-means in eigenspace is relatively fast.

Normalized Cut vs Ratio Cut.

- Both require the same eigen-decomposition cost.
- Normalized cut converges in fewer k-means iterations due to clean embedding.
- Ratio cut requires more iterations and is less stable, increasing computation.

In practice, normalized cut achieves the best trade-off between accuracy and efficiency.

3.3 Additional Observations (4%)

- The kernel hyperparameters (γ_s, γ_c) greatly influence segmentation: too small values blur boundaries, while too large values cause over-segmentation.
- Increasing k beyond 4 or 5 tends to produce visually cluttered regions for all methods.
- Eigenspace visualizations reveal deep insights into algorithm behavior and failure points.
- Normalized cut is robust not because it finds better eigenvectors, but because it scales the Laplacian to correct for degree irregularities in real images.

4 Conclusion

This homework explored three clustering approaches—kernel k-means, ratio cut, and normalized cut— applied to two images using a combined spatial + color similarity kernel. Through visualizations, GIF animations, and eigenspace analysis, we observed substantial differences in stability, segmentation quality, and computational efficiency.

- **Kernel k-means** can produce reasonable segmentations but is sensitive to initialization and hyperparameters, and tends to generate noisy boundaries.
- **Ratio cut** suffers from distorted eigenspaces, leading to poor clustering performance and high instability, especially for more complex images.
- **Normalized cut** consistently delivers the most reliable and meaningful segmentations. The embeddings are clean, continuous, and reflective of underlying image structure.

Overall, my experiments strongly validate the theoretical advantages of the normalized Laplacian in spectral clustering. The method provides the best balance of segmentation quality, robustness, and convergence behavior, making it the most suitable approach among those tested for image segmentation tasks.